

TP2 Explicando las diferencias

Ejercicio 1 | El jugador más "retador" y el jugador más "retado"

En el **TP1**, el código se hizo con MapReduce usando `fmap` y `fred`, donde había que programar todo a mano: cómo leer los datos, generar las claves, agrupar y contar. Era más largo, menos claro y con muchas partes que el programador debía controlar directamente.

En el **TP2**, con PySpark, todo eso se simplifica. Las operaciones se hacen con RDDs y DataFrames, y Spark se encarga del trabajo pesado (distribuir, agrupar y reducir). El código es mucho más corto, más fácil de leer y de mantener.

Ejercicio 2 | El jugador que más puntos obtuvo en promedio

En el **TP1**, el cálculo del promedio se hizo con el modelo MapReduce, definiendo a mano las funciones `fmap2` y `fred2`. En el *mapper* se tomaba el ID del retador y su puntaje, y en el *reducer* se sumaban los puntos y combates para luego calcular el promedio con la fórmula ajustada. Después se leía el archivo de salida para buscar el jugador con mejor promedio. Todo el flujo —lectura, conteo, reducción y salida— se manejaba de forma manual, con más código y pasos intermedios.

En el **TP2**, usando PySpark, el proceso es mucho más directo. Se leen los datos con `sc.textFile()`, se mapean los pares `(id_retador, (puntaje, 1))`, se agrupan con `reduceByKey()` y se calcula el promedio con `mapValues()`. Finalmente, con `max()` se obtiene el jugador con el promedio más alto. Spark maneja internamente toda la distribución y sincronización.

Ejercicio 3 | Todos los jugadores que “retaron” a más de H oponentes distintos

En el **TP1**, este punto se resolvió con un enfoque clásico de MapReduce, donde se definieron las funciones `fmap3` y `fred3`. En el *mapper* se emitía el par `(id_retador, id_retado)` por cada línea, y en el *reducer* se reunían todos los oponentes de cada jugador, usando un conjunto (`set`) para contar solo los distintos. Luego se filtraban los jugadores cuya cantidad de oponentes superaba el valor del parámetro **H**. Este enfoque funciona bien, pero requiere escribir más código y manejar manualmente el flujo de datos y el archivo de salida.

En el **TP2**, la misma lógica se implementó con PySpark de forma mucho más simple. Con unas pocas transformaciones (`map`, `groupByKey`, `mapValues`, `filter` y `collect`) se logró el mismo resultado. Spark se encarga internamente de la agrupación y del manejo de datos distribuidos, lo que hace que el código sea más limpio, corto y fácil de leer.

Ejercicio 4 | El top 10 de los jugadores con mejor puntaje heroico

En el **TP1**, el cálculo del **Puntaje Heroico (PH)** se hizo usando tres *jobs* separados dentro del modelo MapReduce: uno para filtrar los datos, otro para calcular el promedio de puntajes (PP) y un tercero para iterar sobre el cálculo del PH. Cada *job* tiene sus propias funciones `fmap` y `fred`, y es el programador quien debe manejar manualmente la lectura y escritura de archivos, los parámetros compartidos entre etapas y el control de las iteraciones.

Esto hace que el código sea mucho más largo y con muchos pasos intermedios, aunque también brinda control total sobre

cada fase del proceso.

En el **TP2**, el mismo proceso se implementó con **PySpark**, donde los tres *jobs* se convierten en un solo flujo continuo.

Las transformaciones sobre DataFrames (`groupBy` , `agg` , `join` , `fillna` , etc.) reemplazan la lógica de *map* y *reduce*, y Spark se encarga automáticamente de toda la paralelización, la distribución de datos y las iteraciones internas.

El código es mucho más legible, las operaciones son más declarativas y no hace falta manejar archivos de salida ni contexto manualmente.